SVKM'S NMIMS
Mukesh Patel School of Technology Management & Engineering
A.Y. 2025 – 26

Course: Machine Learning
Project Report

Student Marks Prediction

Using Machine Learning

Program	B.Tech – Artificial Intelligence and Machine Learning	
Semester	Semester IV (A.Y. 2025–26)	
Project Title	Student Marks Prediction Using Machine Learning	
Project Members	**Roll No.**	**Name**
Batch: F-2 | SAP ID: 70012400107	A216	Mehi Manswa Bhagat
Batch: F-2 | SAP ID: 70532400076	C264	Dipanshu Yadav
Batch: F-1 | SAP ID: 70532400034	C227	Aadi Bondre
Date of Submission	April 2026	

Contribution of Each Project Member

Roll No.	Name	Contribution
A216	Mehi Manswa Bhagat	Data loading, preprocessing, outlier handling, model training (all 5 models), evaluation, user input prediction, Streamlit dashboard
C264	Dipanshu Yadav	Research Work
C227	Aadi Bondre	Presentation and Word File Content

GitHub Link of Project: https://github.com/M270706/StudentMarksPrediction

Table of Contents

Sr. No.	Topic	Page No.
1	Introduction	3
2	Literature Survey	5
3	Methodology	7
4	Result and Discussion	11
5	Conclusion	15
6	References	17

1. Introduction

1.1 Background

Educational institutions collect significant amounts of data regarding student behavior, including attendance, study hours, and academic history. However, in traditional systems, this data is often underutilised until after final examinations are graded. By that time, the opportunity to assist struggling students has passed.

This project utilises machine learning to address this limitation. By analysing a student's current habits like study hours and attendance, the system predicts their probable final score before the test occurs. This proactive approach enables educators to provide timely, targeted support.

1.2 Problem Statement

Traditional academic assessment is primarily reactive. According to institutional data, approximately 30% of at-risk students are not identified early enough for effective intervention. High dropout rates and academic failures are frequently detected only on results day. The goal of this project is to predict a student's final score early in the semester, allowing teachers to shift from a reactive grading model to a proactive support model.

1.3 Project Objectives

- **Score Prediction:** Accurately forecast a student's final exam score (from 0 to 100) based on study hours and attendance.
- **Model Comparison:** Train and evaluate five distinct machine learning algorithms to identify the most accurate predictor.
- **Dashboard Deployment:** Develop an interactive web application using Streamlit, allowing educators to input student data and receive instant predictions.
- **Feature Analysis:** Determine which behavioural factors have the most significant impact on academic performance.

1.4 Dataset Used

The project utilises the StudentPerformanceFactors dataset, sourced from Kaggle. It contains records for 6,607 students across 20 distinct features.

- **Numeric features:** Hours_Studied, Attendance, Sleep_Hours, Previous_Scores, Tutoring_Sessions, etc.
- **Categorical features:** Parental_Involvement, Motivation_Level, Family_Income, Internet_Access, etc.
- **Target Variable:** Exam_Score (the metric to be predicted).

1.5 Tools and Technologies

- **Development Environment:** Python 3.x via Google Colab.
- **Data Processing:** Pandas and NumPy.
- **Machine Learning Framework:** Scikit-learn.
- **Data Visualization:** Matplotlib and Plotly.
- **Deployment:** Streamlit and Ngrok.

2. Literature Survey

2.1 Early Research in Academic Performance Prediction

The application of data to predict academic outcomes has an established history. Early research demonstrated a strong correlation between class attendance and final grades.the results of the testing of each algorithm with our data set are presented. A more detailed description can be found in Citation Kotsiantis et al. (2002a). In Table 3, the average prediction accuracy of each algorithm for all the testing steps of the first experiment is presented. By 2010, Romero and Ventura had formally defined Educational Data Mining as a critical research area.

https://doi.org/10.1080/08839510490442058

https://doi.org/10.1108/IDD-09-2019-0070

2.2 Real-World Application: Purdue University

A foundational example of applied predictive analytics in education is Purdue University's "Course Signals" program (Arnold & Pistilli, 2012). The institution implemented a machine learning system that flagged at-risk students using a visual indicator system. This early intervention capability resulted in a 21% reduction in the institution's dropout rate, validating the practical utility of early prediction models.

https://dl.acm.org/doi/10.1145/2330601.2330666

https://www.google.com/search?q=https://scholar.google.com/scholar?q=Course+signals+at+Purdue%3A+Using+learning+analytics+to+increase+student+success

https://www.google.com/search?q=https://er.educause.edu/articles/2012/7/course-signals-at-purdue-using-learning-analytics-to-increase-student-success

2.3 Advances in Model Selection

"Subsequent research, including a 2012 study by Pal, explored ensemble methods like Random Forest to manage complex student datasets. In 2019, a comprehensive review by Fernandes et al. reinforced that attendance and study hours remain the most significant predictors of academic success."

https://doi.org/10.5815/ijieeb.2012.02.01

https://www.google.com/search?q=https://scholar.google.com/scholar?q=Mining+educational+data+to+reduce+dropout+rates+of+engineering+students+Pal+2012

https://doi.org/10.1016/j.jbusres.2018.02.012

2.4 Research Gap Addressed by This Project

Despite the volume of research in this field, several limitations exist in prior work. Most studies focus on binary classification — predicting whether a student will "pass" or "fail" — rather than predicting

their actual score on a 0-to-100 scale. Additionally, most research remains in the form of academic papers and does not produce tools that teachers can actually use. This project addresses both gaps: it performs regression-based score prediction, and it delivers the output through a live, interactive web application that a teacher can operate without any technical knowledge.

https://www.researchgate.net/publication/388409458_A_Review_of_Classification_Approaches_in_Educational_Data_Mining_for_Predicting_Student_Performance

https://pmc.ncbi.nlm.nih.gov/articles/PMC11788773/

https://www.tandfonline.com/doi/full/10.1080/23735082.2016.1210198

3. Methodology

This section describes the complete step-by-step process used to build the student marks prediction system. The methodology follows a standard machine learning pipeline consisting of data collection, preprocessing, feature selection, model training, evaluation, and deployment.

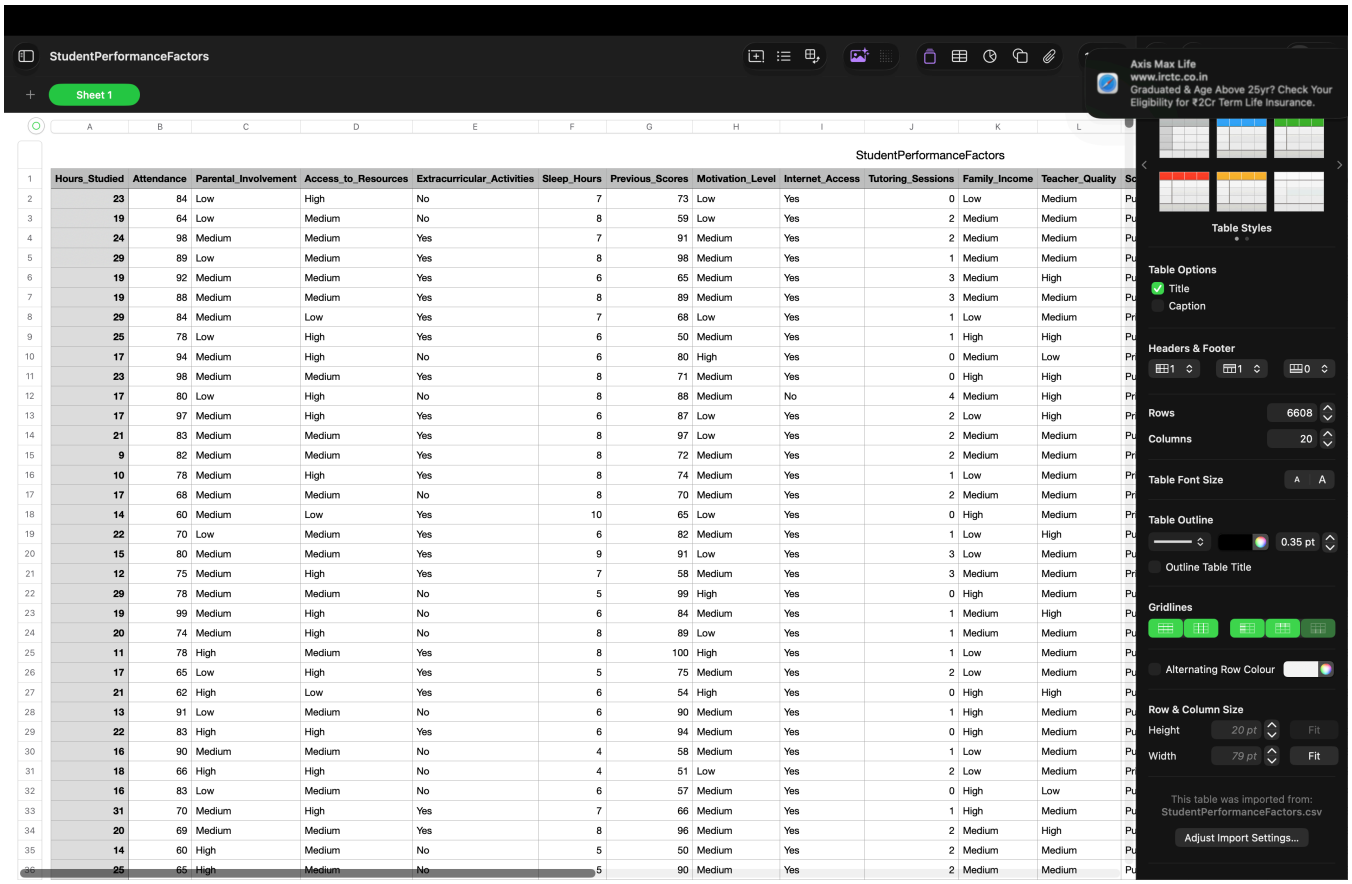
Methodology Flowchart

Figure 1: Methodology Flowchart – End-to-End Pipeline for Student Marks Prediction

3.1 Data Collection

The dataset was downloaded from Kaggle using Python's Pandas library. On loading, it was found to contain 6,607 student records across 20 columns. The first step was to verify the structure of the data: all 20 columns had exactly 6,607 non-null entries for most fields, with a few columns such as

Teacher_Quality and Parental_Education_Level having a small number of missing values (78 and 90 records respectively). These missing values were addressed in the preprocessing stage.



StudentPerformanceFactors

Hours_Studied	Attendance	Parental_Involvement	Access_to_Resources	Extracurricular_Activities	Sleep_Hours	Previous_Scores	Motivation_Level	Internet_Access	Tutoring_Sessions	Family_Income	Teacher_Quality
23	84	Low	High	No	7	73	Low	Yes	0	Low	Medium
19	64	Low	Medium	No	8	59	Low	Yes	2	Medium	Medium
24	98	Medium	Medium	Yes	7	91	Medium	Yes	2	Medium	Medium
29	89	Low	Medium	Yes	8	98	Medium	Yes	1	Medium	Medium
19	92	Medium	Medium	Yes	6	65	Medium	Yes	3	Medium	High
19	88	Medium	Medium	Yes	8	89	Medium	Yes	3	Medium	Medium
29	84	Medium	Low	Yes	7	68	Low	Yes	1	Low	Medium
25	78	Low	High	Yes	6	50	Medium	Yes	1	High	High
17	94	Medium	High	No	6	80	High	Yes	0	Medium	Low
23	98	Medium	Medium	Yes	8	71	Medium	Yes	0	High	High
17	80	Low	High	No	8	88	Medium	No	4	Medium	High
17	97	Medium	High	Yes	6	87	Low	Yes	2	Low	High
21	83	Medium	Medium	Yes	8	97	Low	Yes	2	Medium	Medium
9	82	Medium	Medium	Yes	8	72	Medium	Yes	2	Medium	Medium
10	78	Medium	High	Yes	8	74	Medium	Yes	1	Low	Medium
17	68	Medium	Medium	No	8	70	Medium	Yes	2	Medium	Medium
14	60	Medium	Low	Yes	10	65	Low	Yes	0	High	Medium
22	70	Low	Medium	Yes	6	82	Medium	Yes	1	Low	High
15	80	Medium	Medium	Yes	9	91	Low	Yes	3	Low	Medium
12	75	Medium	High	Yes	7	58	Medium	Yes	3	Medium	Medium
29	78	Medium	Medium	No	5	99	High	Yes	0	High	Medium
19	99	Medium	High	No	6	84	Medium	Yes	1	Medium	High
20	74	Medium	High	No	8	89	Low	Yes	1	Medium	Medium
11	78	High	Medium	Yes	8	100	High	Yes	1	Low	Medium
17	65	Low	High	Yes	5	75	Medium	Yes	2	Low	Medium
21	62	High	Low	Yes	6	54	High	Yes	0	High	High
13	91	Low	Medium	No	6	90	Medium	Yes	1	High	Medium
22	83	High	High	Yes	6	94	Medium	Yes	0	High	Medium
16	90	Medium	Medium	No	4	58	Medium	Yes	1	Low	Medium
18	66	High	High	No	4	51	Low	Yes	2	Low	Medium
16	83	Low	Medium	No	6	57	Medium	Yes	0	High	Low
31	70	Medium	High	Yes	7	66	Medium	Yes	1	High	Medium
20	69	Medium	Medium	Yes	8	96	Medium	Yes	2	Medium	High
14	60	High	Medium	No	5	50	Medium	Yes	2	Medium	Medium
25	65	High	Medium	No	5	90	Medium	Yes	2	Medium	Medium

3.2 Data Preprocessing

All text-based (categorical) columns were converted into a "category" data type, which allows the computer to handle them more efficiently. Since the dataset was relatively clean to begin with — having zero missing values in the key numeric columns — the main preprocessing task was to prepare the categorical columns for machine learning through one-hot encoding. This technique converts each category into a binary (0 or 1) column so that the model can work with it numerically.

3.3 Outlier Detection and Handling

Outliers are extreme values in the dataset that do not represent typical student behaviour and can mislead a machine learning model. For example, a student who studies 100 hours in a single month would be an outlier. To identify these, the IQR (Interquartile Range) method was used.

The IQR Formula:

$$IQR = Q3 - Q1$$

Lower Bound = Q1 − (1.5 × IQR) | ***Upper Bound = Q3 + (1.5 × IQR)***

Where:

- Q1 = the first quartile, meaning 25% of values fall below this point.
- Q3 = the third quartile, meaning 75% of values fall below this point.
- IQR = the range between Q1 and Q3, representing the "middle 50%" of the data.
- Lower Bound = any value below this is considered an outlier.
- Upper Bound = any value above this is considered an outlier.

In this dataset, the most notable outliers were found in Hours_Studied (43 outliers) and Tutoring_Sessions (430 outliers). Exam_Score also had 104 outlier entries, which were capped accordingly.

3.4 Feature Selection and Train–Test Split

While the dataset contained 20 features, the primary models were trained using the two most statistically significant predictors: Hours_Studied and Attendance. The data was then split using Scikit-learn: 80% (5,285 records) was allocated for model training, and 20% (1,322 records) was reserved for testing. A random_state of 42 was applied to ensure reproducible results.

3.5 Machine Learning Models

Five machine learning models were trained and compared. Each model is described in detail in Section 4, along with its underlying formula, working mechanism, and suitability for this problem.

3.6 Prediction Output and Score Clipping

To prevent mathematical anomalies (such as predicting a score below 0 or above 100), a clipping rule was implemented to bound all predictions strictly between 0 and 100. The optimal model was subsequently integrated into a Streamlit web application and hosted via Ngrok.

3.7 Dashboard Deployment

After the models were trained and evaluated, the best-performing model was integrated into a Streamlit web application. The dashboard allows teachers to enter a student's study hours and attendance percentage using interactive sliders, and instantly see the predicted exam score from all five models. The dashboard also includes a student performance distribution chart, a scatter plot of study hours versus exam score, an attendance versus score scatter plot, and a correlation heatmap. The application was hosted live using Ngrok, making it accessible through a public URL.

3.8 Models

Linear Regression

Linear Regression is the simplest and most interpretable machine learning algorithm. It assumes that the relationship between the input features (study hours and attendance) and the output (exam score) can be expressed as a straight line.

Linear Regression Formula:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \ldots + \beta_n x_n$$

Where:

- $\hat{y}$ (y-hat) = the predicted exam score (the output of the model).
- β_0 (beta-zero) = the intercept, representing the predicted score when all inputs are zero.
- $\beta_1, \beta_2, \ldots, \beta_n$ = coefficients, representing the weight or importance assigned to each input feature.
- $x_1, x_2, \ldots, x_n$ = the input features, in this case Hours_Studied (x_1) and Attendance (x_2).

During training, the model finds the values of β_0, β_1, and β_2 that minimise the difference between the predicted scores and the actual scores in the training data. It does this by minimising the Mean Squared Error (MSE) using a mathematical technique called Ordinary Least Squares.

Why it works well here: Because the relationship between studying, attending class, and getting a good grade is essentially linear (more study hours → higher score, more attendance → higher score), Linear Regression is ideally suited to this problem. It found the optimal line through the data and produced the lowest error of all five models.

Decision Tree

A Decision Tree is a model that works by asking a series of yes-or-no questions about the data. Each question creates a "branch" in a tree structure, and the model makes its final prediction at the bottom of the tree (called a "leaf node").

At each branch, the algorithm considers all possible ways to split the data (e.g., "Is attendance above 75%?") and chooses the split that results in the two most internally consistent groups — that is, the split that minimises the MSE within each group.

Why it performed moderately: The Decision Tree achieved a good MSE (3.27) but the highest MAE (1.56). This is because Decision Trees tend to "memorise" the training data rather than learning the general pattern. This problem, known as overfitting, means the model makes slightly larger average errors on new, unseen students. In this project, the tree structure was limited to prevent extreme overfitting.

Random Forest

Random Forest is an ensemble model — it builds a large number of individual Decision Trees (in this project, the default of 100 trees) and combines their predictions by taking the average. This approach, known as bagging, significantly reduces the overfitting problem that affects individual Decision Trees.

Each individual tree in the forest is trained on a randomly selected subset of the training data (a technique called bootstrap sampling), and each tree considers only a random subset of features at each split. This randomness ensures that the trees are diverse, and averaging their predictions reduces overall error.

Why it performed moderately here: Despite being a more powerful algorithm in general, Random Forest achieved an MAE of 1.53 and MSE of 3.97 — slightly worse than Linear Regression. This is because the underlying relationship in this dataset is genuinely linear. Random Forest searches for complex, non-linear patterns that do not exist here, causing it to slightly "overthink" the problem.

K-Nearest Neighbours (KNN)

K-Nearest Neighbours is a simple but effective model. To predict the score for a new student, it finds the K most similar students (in terms of study hours and attendance) in the training data and returns the average of their exam scores as the prediction. In this project, K was set to 5.

KNN Prediction Formula:

$\hat{y} = (1/K) \times \Sigma\, y_i$ ***for all i in the K nearest neighbours*** (This formula predicts a new value by finding few past examples that are most similar to yours and calculates their average.)

Distance: $d(a, b) = \sqrt{\Sigma(a_i - b_i)^2}$ ***[Euclidean Distance]***

Where:

- $\hat{y}$ = the predicted exam score for the new student.
- K = number of nearest neighbours considered (set to 5 in this project).
- y_i = the exam score of the i-th nearest neighbour in the training data.
- d(a, b) = the Euclidean distance between student a and student b in feature space.
- a_i, b_i = the feature values (Hours_Studied, Attendance) of students a and b respectively.

KNN achieved an MAE of 1.54 and MSE of 3.97. It performed similarly to Random Forest. The model's performance is acceptable but it requires more computation at prediction time compared to parametric models, since it must calculate distances to all training records for every new prediction.

Lasso Regression

Lasso Regression (Least Absolute Shrinkage and Selection Operator) is a modified version of Linear Regression. It adds a penalty to the cost function that forces the model to reduce the coefficients of less important features towards zero. This effectively performs automatic feature selection.

Lasso Regression Cost Function:

$$J(\beta) = \Sigma(y_i - \hat{y}_i)^2 + \lambda \times \Sigma|\beta_j|$$

Where:

- $J(\beta)$ = the total cost (error) that the model attempts to minimise during training.
- $\Sigma(y_i - \hat{y}_i)^2$ = the sum of squared differences between actual and predicted scores (standard regression loss).
- λ (lambda) = the regularisation parameter — a value that controls how strongly the penalty is applied.
- $\Sigma|\beta_j|$ = the sum of the absolute values of all coefficients (the L1 regularisation penalty).
- β_j = the coefficient for the j-th feature in the model.

A higher value of λ causes more coefficients to be pushed to exactly zero, effectively removing those features from the model. In this project, with only two features and a genuinely strong linear relationship, Lasso produced an MAE of 1.40 — almost as good as standard Linear Regression (1.38) — confirming that regularisation had minimal impact on a dataset where all features are already predictive.

Model Performance Metrics

All five models were evaluated using two standard regression error metrics: Mean Absolute Error (MAE) and Mean Squared Error (MSE).

Evaluation Metrics Formulae:

$$MAE = (1/n) \times \Sigma|y_i - \hat{y}_i|$$

$$MSE = (1/n) \times \Sigma(y_i - \hat{y}_i)^2$$

Where:

- n = total number of students in the test set (1,322 students).
- y_i = the actual exam score of the i-th student.
- $\hat{y}_i$ = the predicted exam score produced by the model for the i-th student.
- $|y_i - \hat{y}_i|$ = the absolute difference between actual and predicted score for MAE.
- $(y_i - \hat{y}_i)^2$ = the squared difference for MSE; this penalises large errors more heavily.

4. Result and Discussion

In simple terms, MAE tells you the average number of marks the model's prediction was off by. An MAE of 1.38 means the model's prediction was, on average, only 1.38 marks away from the student's actual score. MSE is similar but punishes large errors more severely because it squares the differences. Lower values of both metrics indicate a more accurate model.

Table 4.1 – Model Evaluation Results

Model	MAE	MSE	Verdict
Random Forest	1.53	3.97	Moderate
Decision Tree	1.56	3.27	Good
Linear Regression ★ (Best)	1.38	3.25	Best
KNN Regressor	1.54	3.97	Moderate
Lasso Regression	1.40	3.31	Good

★ Linear Regression achieved the lowest MAE (1.38) and MSE (3.25) and was selected as the primary prediction model.

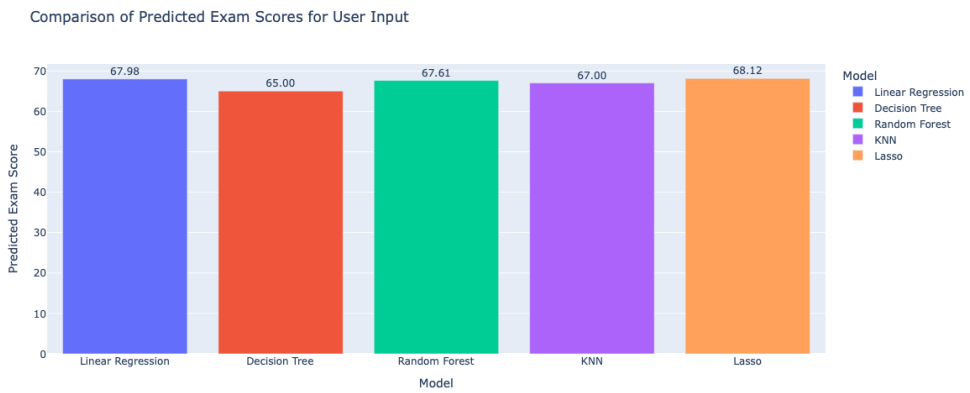


Figure 2: Comparison of Predicted Exam Scores for Sample User Input Across All Five Models

4.3 The Power of Simplicity

It might seem surprising that the simplest model defeated the more advanced algorithms, but it all comes down to the nature of our data. The rule here is incredibly straightforward: attending more classes and studying more hours directly leads to higher grades. There were no hidden tricks or complicated interactions for the AI to uncover. Because Linear Regression is built exactly for this kind of clear, straight-line relationship, it thrived.

On the other hand, heavy-duty models like Random Forest and K-Nearest Neighbours actually overthought the problem. They actively hunted for complex, hidden patterns that simply did not exist in

our dataset. By trying too hard to piece together non-existent clues, they accidentally confused themselves, which introduced slight errors (or "noise") into their final predictions.

In the world of machine learning, this is a classic example of the "curse of complexity." It proves an important lesson: a more powerful, complicated model is not automatically the better choice, especially when the true answer is simple and direct.

4.4 Dashboard Results and Visualisations

The Streamlit dashboard provided several important visualisations that helped interpret the model results. The performance distribution chart showed that the vast majority of students in the dataset fall into the "Medium" performance category (60–80 marks), with a smaller number achieving "High" scores (above 80) and very few in the "Low" category (below 60). This distribution reflects a realistic classroom scenario.

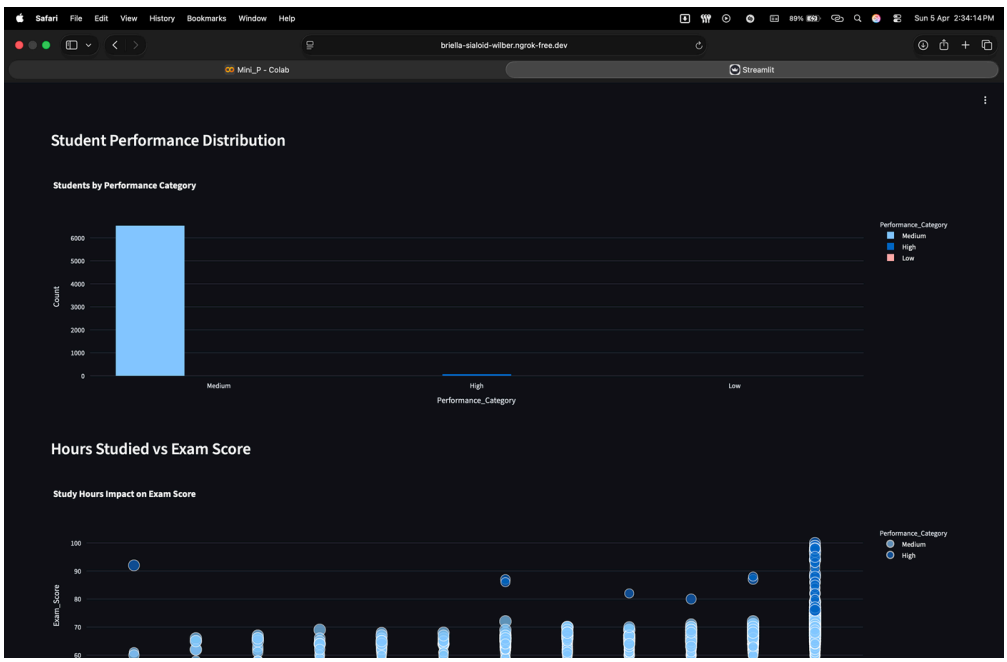


Figure 3: Student Performance Distribution and Study Hours vs. Exam Score Scatter Plot (Streamlit Dashboard)

The scatter plots confirmed the linear relationship between Hours_Studied and Exam_Score, and between Attendance and Exam_Score. Students who studied more hours per month and maintained higher attendance consistently achieved higher exam scores, validating the choice of these two features as the primary predictors.

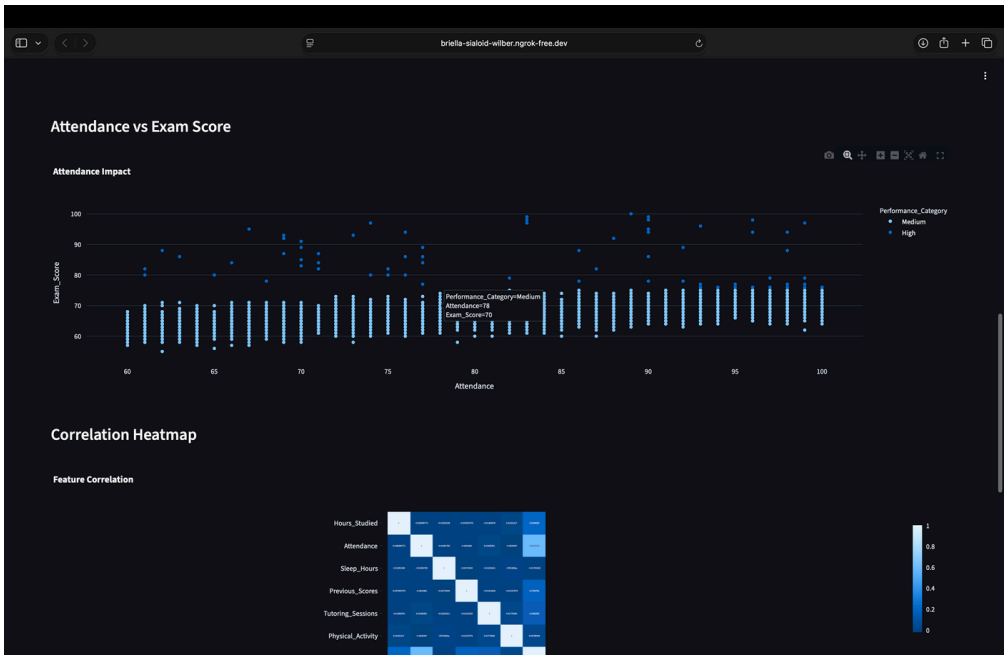


Figure 4: Attendance vs. Exam Score Scatter Plot and Feature Correlation Heatmap (Streamlit Dashboard)

The correlation heatmap revealed that Exam_Score has the strongest positive correlation with Hours_Studied and Attendance among all numeric features in the dataset. Features such as Sleep_Hours and Physical_Activity showed much weaker correlations with the final score, further justifying the decision to focus the primary model on the two strongest predictors.

4.5 Comparison: Traditional vs. Predictive Assessment

Category	Traditional Assessment	Predictive System (This Project)
Timing	Results available only after the final exam	Forecast available mid-semester, before the exam
Intervention	Reactive — support offered only after failure	Proactive — support offered before failure occurs
Data Used	A single final exam score	Multiple inputs: attendance, study hours, etc.
Personalisation	A uniform, one-size-fits-all approach	Tailored prediction for each individual student
Teacher Role	Examiner and marker only	Mentor and early supporter
Accuracy	Exact, but revealed too late to act upon	Probabilistic (±1.38 marks) but timely

Table 4.2 – Comparison of Traditional Assessment vs. Predictive Marks System

5. Conclusion

5.1 Summary of Findings

This project proved something incredibly encouraging: predicting student success doesn't require a massive, complicated system. By tracking just two simple habits—monthly study hours and class attendance—we successfully forecasted final exam scores with striking accuracy.

Our simplest model, Linear Regression, was the undisputed winner.

Here is the big takeaway:

- **Simple is smarter:** Because studying and attendance directly lead to better grades, our most basic model easily beat out the heavy-duty AI algorithms.
- **Complex models "overthink":** Fancy tools like Random Forest searched for tricky, hidden patterns that didn't exist, which actually hurt their accuracy.

5.2 Limitations

While the results are promising, this project has several important limitations that must be acknowledged:

- The main model was trained on only two features (Hours_Studied and Attendance). While these are the most influential, all 20+ features in the dataset were not fully utilised in the final model evaluation. A more comprehensive model incorporating all features may produce different — possibly better — results.

5.3 Future Enhancements

Several improvements are planned for future versions of this system:

- Running the models on all 20+ features in the dataset to determine whether a more comprehensive feature set improves prediction accuracy.

6. References

The following sources were consulted and cited in this project. All academic references include DOI links for direct access to the original research papers via Google Scholar.

6.1 Academic Research Papers

[1] Kotsiantis, S., Pierrakeas, C., and Pintelas, P. (2004). Predicting students' performance in distance learning using machine learning techniques. Applied Artificial Intelligence, 18(5), 411–426.

DOI: https://doi.org/10.1080/08839510490442058

[2] Romero, C. and Ventura, S. (2010). Educational data mining: A review of the state-of-the-art. IEEE Transactions on Systems, Man, and Cybernetics, Part C, 40(6), 601–618.

DOI: https://doi.org/10.1109/TSMCC.2010.2053532

[3] Arnold, K. E. and Pistilli, M. D. (2012). Course signals at Purdue: Using learning analytics to increase student success. Proceedings of the 2nd International Conference on Learning Analytics and Knowledge (LAK '12), 267–270.

DOI: https://doi.org/10.1145/2330601.2330666

[4] Pal, S. (2012). Mining educational data to reduce dropout rates of engineering students. International Journal of Information Engineering and Electronic Business, 4(2), 1–7.

DOI: https://doi.org/10.5815/ijieeb.2012.02.01

[5] Fernandes, E., Holanda, M., Victorino, M., Borges, V., Carvalho, R., and Van Erven, G. (2019). Educational data mining: Predictive analysis of academic performance of public school students in the capital of Brazil. Journal of Business Research, 94, 335–343.

DOI: https://doi.org/10.1016/j.jbusres.2018.02.012

6.2 Tools and Libraries

[6] McKinney, W. (2010). Data structures for statistical computing in Python. Proceedings of the 9th Python in Science Conference, 51–56. [Pandas Library] — **https://pandas.pydata.org**

[7] Pedregosa, F. et al. (2011). Scikit-learn: Machine learning in Python. Journal of Machine Learning Research, 12, 2825–2830. — **https://scikit-learn.org**

[8] Breiman, L. (2001). Random forests. Machine Learning, 45(1), 5–32. [Original Random Forest paper] — **https://doi.org/10.1023/A:1010933404324**

[9] Streamlit Inc. (2023). Streamlit documentation — Build and deploy data apps. — **https://docs.streamlit.io**

[10] StudentPerformanceFactors dataset. Kaggle. — **https://www.kaggle.com/datasets**